

Redes Neuronales Recurrentes Análogas con Pesos Enteros

Andrés Sicard Ramírez
asicard@eafit.edu.co

Juan C. Agudelo Agudelo
jagudelo@eafit.edu.co

Mario E. Vélez Ruiz
mvelez@eafit.edu.co

Grupo de Lógica y Computación
Escuela de Ciencias y Humanidades
Universidad EAFIT; Medellín, Colombia

Resumen

Se definen las redes neuronales recurrentes análogas (ARNN) propuestas por Hava T. Siegelmann y Eduardo Sontag [6]. Se definen las máquinas de estado finito y se demuestra la equivalencia computacional entre las ARNN con pesos enteros y las máquinas de estado finito.

Abstract

We define analog recurrent neural networks (ARNN) proposed by Hava T. Siegelmann and Eduardo Sontag [6], we define finite state machines and then we demonstrate computational equivalence between ARNNs with integer weights and finite state machines.

1 Introducción

Una red neuronal es una interconexión de procesadores simples o “*neuronas*”. Las conexiones entre las neuronas son dirigidas y caracterizadas por números llamados pesos. En cada instante discreto de tiempo, la red neuronal recibe unas entradas del mundo exterior y cada neurona computa su valor de activación aplicando una función sobre las entradas: los pesos y valores de las conexiones entrantes y los valores de activación de las neuronas en el instante anterior. La salida de la red neuronal son los valores de activación de un subconjunto determinado de neuronas.

Las redes neuronales se pueden clasificar de acuerdo a su arquitectura en “*feedforward*” (de alimentación hacia adelante o por niveles, fig. 1) y “*feedback*” (de alimentación hacia atrás o recurrentes, fig. 2). En las primeras, las neuronas están organizadas en múltiples niveles, de tal forma que las salidas de un nivel son las entradas del siguiente, por lo tanto, el grafo de interconexión de la red es acíclico. En las segundas, se permiten ciclos en sus grafos de interconexión [6].

2 Redes Neuronales Recurrentes Análogas

Las ARNN (*Analog Recurrent Neural Networks*) definidas por Hava Siegelmann son redes neuronales del tipo “*feedback*” donde se permite que los pesos de las conexiones sean números reales.

Definición 1 (Red Neuronal Recurrente Análoga). *Una ARNN está compuesta por n procesadores elementales llamados neuronas, cada neurona tiene asociado un valor de activación temporal, estos valores son*

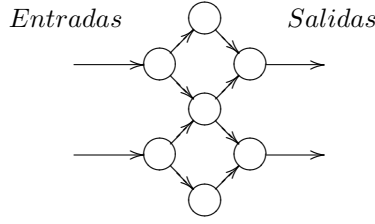


Figura 1: Red neuronal tipo “feedforward”.

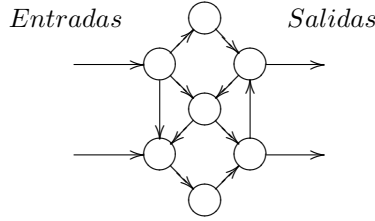


Figura 2: Red neuronal tipo “feedback”.

representados por el vector de activación $X(t)$, de dimensión $n \times 1$. Además, cada neurona tiene asociado un peso constante, estos pesos son representados por el vector C , de dimensión $n \times 1$. Las neuronas están conectadas por medio de enlaces dirigidos cada uno de los cuales tiene asociado un peso, estos son representados por medio de la matriz A , de dimensión $n \times n$, donde el elemento a_{ij} es el peso del enlace que va de la neurona x_j a la x_i si existe tal enlace o cero si no existe, de este modo la fila i contiene los pesos de los enlaces que entran a la neurona x_i . En cada instante discreto de tiempo la red recibe un conjunto de m entradas binarias, representadas por el vector de entradas $U(t)$, de dimensión $m \times 1$, entre estas entradas y las neuronas existen enlaces dirigidos, los cuales también tiene pesos asociados y son representados por la matriz B , de dimensión $n \times m$, donde el elemento b_{ij} es el peso del enlace entre la entrada u_j y la neurona x_i si tal enlace existe o cero de otro modo, la fila i representa entonces los pesos de las entradas a la neurona x_i . Cada que la red recibe un nuevo conjunto de entradas recalcula el valor de activación de las neuronas de acuerdo a la ecuación:

$$X(t+1) = \sigma(A \cdot X(t) + B \cdot U(t) + C), \quad (1)$$

donde σ es una función que se aplica a cada uno de los elementos del vector resultante y es llamada función de activación. La función σ más comúnmente utilizada es:

$$\sigma(x) = \begin{cases} 0 & \text{sii } x < 0, \\ x & \text{sii } 0 \leq x \leq 1, \\ 1 & \text{sii } x > 1. \end{cases} \quad (2)$$

La salida de la red son los valores de activación calculados de un subconjunto determinado de neuronas $y_1(t), y_2(t), \dots, y_l(t)$, donde $y_i(t) \in X(t)$.

Definición 2 (Computación en una ARNN). *Una computación en una ARNN es una sucesión de cálculos en instantes discretos de tiempo de los valores de activación $X(t+1)$ de las neuronas de acuerdo a los valores de activación $X(t)$ y las entradas $U(t)$ en el instante de tiempo anterior, teniendo en cuenta los enlaces y pesos de la red A, B y C , de acuerdo a la ecuación (1). Las salidas en cada iteración son un subconjunto de los valores de activación calculados $y_1(t), y_2(t), \dots, y_l(t)$. Al inicio de la computación, tiempo $t = 0$, cada neurona tiene un valor de activación inicial $x_i(0)$ (normalmente $x_i(0) = 0$ para $i = 1, 2, \dots, N$). La computación termina cuando no hay más entradas.*

Hay que tener en cuenta que en la mayoría de ARNN se introduce un retardo r en la computación, es decir, los primeros r valores de activación de las neuronas de salida no deben ser tomados en cuenta como parte del cómputo, esto se debe a que las entradas por lo general no llegan directamente a las neuronas de salida y para llegar a afectarlas tienen primero que propagarse por otras neuronas de la red, este tiempo de retardo depende de la estructura específica de la red y será especificado de modo informal. Además, hay que tener en cuenta que la red debe seguir computando una vez se hallan terminado los datos de entrada, para que los últimos datos ingresados lleguen a afectar las neuronas de salida, para tal efecto se deben ingresar r entradas con valores cero, es decir, $U(t) = 0$ para valores de t mayores que la longitud de la entrada a procesar, los valores cero en estas entradas aseguran que la ARNN siga computando sin ningún problema hasta producir la salida por completo.

Definición 3 (Diagrama de Red). *Una ARNN se puede representar por un grafo dirigido (posiblemente con ciclos) llamado diagrama de red, bajo las siguientes convenciones:*

1. *Nodos: Neuronas $x_i(t) \in X(t)$, etiquetados con x_i .*
2. *Arcos: Existe un arco entre el nodo x_j y el nodo x_i , etiquetado con a_{ij} si y sólo si $a_{ij} \neq 0$.*
3. *Se coloca un arco con extremo inicial u_j , y extremo final el nodo x_i , etiquetado con b_{ij} si y sólo si $b_{ij} \neq 0$.*
4. *Las salidas de la ARNN se representan con arcos saliendo de las neuronas de salida.*
5. *Los pesos constantes $c_i \in C$ se representan colocando un arco que entra a la neurona x_i etiquetado con c_i si y sólo si $c_i \neq 0$.*

Ejemplo 1. *Una ARNN que suma en binario con retardo de un instante de tiempo ($r = 1$) está definida por:*

Neuronas: $x = \{x_1, x_2, x_3, x_4\}$, donde sólo x_4 es neurona de salida.

Matriz de conexiones entre las neuronas:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & -1 & 1 & 0 \end{pmatrix}.$$

Entradas: $u = \{u_1, u_2\}$, donde u_1 y u_2 representan los dígitos a sumar del operando 1 y 2 respectivamente.

Matriz de pesos de entradas:

$$B = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}.$$

Vector de pesos constantes:

$$C = \begin{pmatrix} 0 \\ -1 \\ -2 \\ 0 \end{pmatrix}.$$

Función de activación: función σ definida en la ecuación 2.

El diagrama de red está dado por la figura 3.

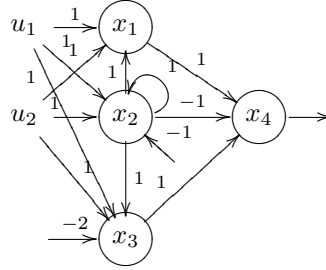


Figura 3: Diagrama de red ARNN de suma binaria.

Para sumar los valores binarios 0101 y 0110 se tiene la siguiente computación:

- Iteración 1:

$$X(1) = \sigma \left(\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & -1 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \\ -2 \\ 0 \end{pmatrix} \right) = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

- Iteración 2:

$$X(2) = \sigma \left(\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & -1 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \\ -2 \\ 0 \end{pmatrix} \right) = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

- Iteración 3:

$$X(3) = \sigma \left(\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & -1 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \\ -2 \\ 0 \end{pmatrix} \right) = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix}.$$

- Iteración 4:

$$X(4) = \sigma \left(\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & -1 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \\ -2 \\ 0 \end{pmatrix} \right) = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

- Iteración 5:

$$X(5) = \sigma \left(\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & -1 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \\ -2 \\ 0 \end{pmatrix} \right) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

Los valores de $x_4(t)$ para $2 \leq t \leq 5$ que aparecen en negrilla: 1, 1, 0, 1 son el resultado de la suma de los dos números.

3 Máquinas de Estado Finito

Una máquina de estado finito es una máquina abstracta. En cada instante de tiempo, la máquina se encuentra en un estado (de un conjunto finito de estados) y recibe un símbolo de entrada, de acuerdo a esto la máquina produce un símbolo de salida y realiza un cambio de estado. Se presenta a continuación una definición formal para tales máquinas [1, 2]:

Definición 4 (Máquina de Estado Finito). Una máquina de estado finito $\mathcal{M}$ es una sextupla:

$$\mathcal{M} = \langle E, S, Q, f, g, q_1 \rangle \text{ donde :} \quad (3)$$

$E: \{e_1, e_2, \dots, e_p\}$ Alfabeto finito de entrada.
 $S: \{s_1, s_2, \dots, s_v\}$ Alfabeto finito de salida.
 $Q: \{q_1, q_2, \dots, q_w\}$ Conjunto finito de estados.
 $f: Q \times E \rightarrow Q$ Función de cambio de estado.
 $g: Q \times E \rightarrow S$ Función de salida.
 $q_1 \in Q$ Estado inicial.

Definición 5 (Tabla de Transición). Una máquina de estado finito se puede representar por medio de una tabla T de transición, siguiendo las siguientes convenciones:

1. Filas: Estados Q .
2. Columnas: Símbolos del alfabeto de entrada E .
3. $T_{i,j} = (q', s')$ si y sólo si $(f(q_i, e_j) = q' \wedge g(q_i, e_j) = s')$.
4. Se denota el estado inicial q_1 , subrayando éste en la fila correspondiente a los estados.

Ejemplo 2. Una máquina de estado finito que suma en binario está definida por:

Alfabeto de entrada: $E = \{00, 01, 10, 11\}$, dígitos a sumar.

Alfabeto de salida: $S = \{0, 1\}$, resultados de la suma.

Estados: $Q = \{N, A\}$, N : No acarreo, A : acarreo.

Las funciones de cambio de estado f y de salida g están especificadas por medio de la tabla de transición (tabla 1).

Q/Σ	00	01	10	11
$\underline{N}$	$N, 0$	$N, 1$	$N, 1$	$A, 0$
A	$N, 1$	$A, 0$	$A, 0$	$A, 1$

Cuadro 1: Tabla de transición para un sumador binario.

Para sumar los valores binarios 0101 y 0110 se tienen las entradas 10, 01, 11 y 00 y se producen las salidas 1, 1, 0, 1 que son el resultado de la suma de los dos números.

Definición 6 (Diagrama de transición). Una máquina de estado finito se puede representar por medio de un grafo dirigido, llamado diagrama de transición, siguiendo las siguientes convenciones:

1. Nodos: $q_i \in Q$.
2. Arcos: Existe un arco del nodo q_i al nodo q_j , etiquetado con e/s , si y sólo si, $f(q_i, e) = q_j$ y $g(q_i, e) = s$.
3. Se coloca un arco no etiquetado para indicar el estado inicial q_1 .

Observación: Para simplificar el diagrama, cuando existen varios arcos entre un nodo y otro, se dibuja sólo un arco con todas las etiquetas.

Ejemplo 3. La figura 4 representa el diagrama de transición para el sumador binario.

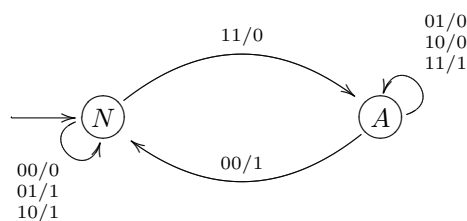


Figura 4: Diagrama de transición para un sumador binario.

4 Relación entre ARNN y Máquinas de Estado Finito

Al restringir algunos parámetros de las ARNN, tales como los posibles valores que pueden tomar los pesos y las características de la función de activación, se pueden obtener modelos con diferente poder de computación. En particular, si se tiene una ARNN con función de activación σ igual a la definida en la ecuación (2), donde σ es igual para todas las neuronas (modelo homogéneo) y se restringe los posibles valores que pueden tomar los pesos de la red a valores enteros, los valores de activación de las neuronas se restringen a valores binarios y la ARNN se vuelve computacionalmente equivalente a las redes neuronales propuestas por McCulloch y Pitts [4], y por consiguiente computacionalmente equivalente a una máquina de estado finito [3, 5, 6]. Donde computacionalmente equivalente quiere decir que tienen el mismo comportamiento entradas-salidas.

Teorema 1. *Toda ARNN con pesos enteros tiene una máquina de estado finito equivalente.*

Demostración. Sea $\mathcal{N}$ una ARNN con vector de activación $X(t)$, vector de entradas $U(t)$, matriz de pesos de conexiones A , matriz de pesos de entradas B , vector de pesos constantes C , función de activación σ (ec. 2) y neuronas de salida $y_1(t), y_2(t), \dots, y_l(t)$, y sean:

- $x_i(t) \in \{0, 1\}$ con $i = 1, 2, \dots, n$ elemento de x .
- $u_i(t) \in \{0, 1\}$ con $i = 1, 2, \dots, m$ elemento de u .
- $a_{ij} \in \mathbb{Z}$ con $i, j = 1, 2, \dots, n$ elemento de A .
- $b_{ij} \in \mathbb{Z}$ con $i = 1, 2, \dots, n; j = 1, 2, \dots, m$ elemento de B .
- $c_i \in \mathbb{Z}$ con $i = 1, 2, \dots, n$ elemento de C .

La máquina de estado finito $\mathcal{M} = \langle E, S, Q, f, g, q_1 \rangle$ equivalente a $\mathcal{N}$ se define por:

- Alfabeto de entrada: $E = \{e_1, e_2, \dots, e_{2^m}\}$ donde e_i es equivalente a la entrada $U(t)$ si y sólo si $1 + \sum_{j=1}^m (2^{j-1} \cdot u_j(t)) = i$.
- Alfabeto de salida: $S = \{s_1, s_2, \dots, s_{2^l}\}$ donde s_i es equivalente a la salida $y_1(t), y_2(t), \dots, y_l(t)$ si y sólo si $1 + \sum_{j=1}^l (2^{j-1} \cdot y_j(t)) = i$.
- Estados: $Q = \{q_1, q_2, \dots, q_{2^n}\}$ donde q_i es equivalente al vector de activación $X(t)$ si y sólo si $1 + \sum_{j=1}^n (2^{j-1} \cdot x_j(t)) = i$.
- Función de cambio de estado: f está determinada por la ecuación (1) así: $f(q_i, e_j) = q_k$ si y sólo si q_i es equivalente a $X(t)$, e_j es equivalente a $U(t)$ y q_k es equivalente a $X(t+1)$.
- Función de salida: g está determinada por la ecuación (1) así: $g(q_i, e_j) = s_k$ si y sólo si q_i es equivalente a $X(t)$, e_j es equivalente a $U(t)$ y s_k es equivalente a $x_{i_1}, x_{i_2}, \dots, x_{i_l}$.
- Estado inicial: q_1 es equivalente a $X(0)$.

□

Ejemplo 4. *Para la ARNN definida en el ejemplo 1, la máquina de estado finito equivalente está definida por:*

Alfabeto de entrada: $E = \{e_1, e_2, e_3, e_4\}$ donde e_i es equivalente a la entrada $U(t)$ de acuerdo a la tabla 2.

$E/U(t)$	u_1	u_2
e_1	0	0
e_2	0	1
e_3	1	0
e_4	1	1

Cuadro 2: Tabla de equivalencias $E/U(t)$.

Alfabeto de salida: $S = \{s_1, s_2\}$, que son equivalentes a los valores de activación 0 y 1 de la neurona de salida x_4 respectivamente.

Estados: $Q = \{q_1, q_2, \dots, q_{16}\}$ donde q_i es equivalente al vector de activación $X(t)$ de acuerdo a la tabla 3.

$Q/X(t)$	x_1	x_2	x_3	x_4
q_1	0	0	0	0
q_2	0	0	0	1
q_3	0	0	1	0
q_4	0	0	1	1
q_5	0	1	0	0
q_6	0	1	0	1
q_7	0	1	1	0
q_8	0	1	1	1
q_9	1	0	0	0
q_{10}	1	0	0	1
q_{11}	1	0	1	0
q_{12}	1	0	1	1
q_{13}	1	1	0	0
q_{14}	1	1	0	1
q_{15}	1	1	1	0
q_{16}	1	1	1	1

Cuadro 3: Tabla de equivalencias $Q/X(t)$.

Tabla de transición de estados: Tabla 4.

De la tabla 4 se puede ver que los estados q_1 y q_2 son equivalentes (las funciones f y g son las mismas para ambos estados), lo mismo que los estados q_3 , q_4 , q_9 y q_{10} , los estados q_5 , q_6 , q_7 , q_8 , q_{13} y q_{14} , y los estados q_{15} y q_{16} . Además, los estados q_{11} y q_{12} son inalcanzables (no existe ninguna transición que llegue a ellos). Agrupando los estados equivalentes y eliminando los estados inalcanzables se obtiene la tabla de transición dada por la tabla 5.

Q/Σ	e_1	e_2	e_3	e_4
q_1	q_1, s_1	q_9, s_1	q_9, s_1	q_{13}, s_1
q_2	q_1, s_1	q_9, s_1	q_9, s_1	q_{13}, s_1
q_3	q_2, s_2	q_{10}, s_2	q_{10}, s_2	q_{14}, s_2
q_4	q_2, s_2	q_{10}, s_2	q_{10}, s_2	q_{14}, s_2
q_5	q_9, s_1	q_{13}, s_1	q_{13}, s_1	q_{15}, s_1
q_6	q_9, s_1	q_{13}, s_1	q_{13}, s_1	q_{15}, s_1
q_7	q_9, s_1	q_{13}, s_1	q_{13}, s_1	q_{15}, s_1
q_8	q_9, s_1	q_{13}, s_1	q_{13}, s_1	q_{15}, s_1
q_9	q_2, s_2	q_{10}, s_2	q_{10}, s_2	q_{14}, s_2
q_{10}	q_2, s_2	q_{10}, s_2	q_{10}, s_2	q_{14}, s_2
q_{11}	q_2, s_2	q_{10}, s_2	q_{10}, s_2	q_{15}, s_1
q_{12}	q_2, s_2	q_{10}, s_2	q_{10}, s_2	q_{15}, s_1
q_{13}	q_9, s_1	q_{13}, s_1	q_{13}, s_1	q_{15}, s_1
q_{14}	q_9, s_1	q_{13}, s_1	q_{13}, s_1	q_{15}, s_1
q_{15}	q_{10}, s_2	q_{14}, s_2	q_{14}, s_2	q_{16}, s_2
q_{16}	q_{10}, s_2	q_{14}, s_2	q_{14}, s_2	q_{16}, s_2

Cuadro 4: Tabla de transición de estados: conversión ARNN ejemplo 1.

Q/Σ	e_1	e_2	e_3	e_4
q_1	q_1, s_1	q_3, s_1	q_3, s_1	q_5, s_1
q_3	q_1, s_2	q_3, s_2	q_3, s_2	q_5, s_2
q_5	q_3, s_1	q_5, s_1	q_5, s_1	q_{15}, s_1
q_{15}	q_3, s_2	q_5, s_2	q_5, s_2	q_{15}, s_2

Cuadro 5: Tabla de transición de estados simplificada: conversión ARNN ejemplo 1.

El diagrama de transición está dado por la figura 5.

Para las entradas $e_3e_2e_4e_1e_1$, que son equivalentes a las entradas que se utilizaron en el ejemplo 1, se obtienen las salidas $s_1s_2s_2s_1s_2$, que son equivalentes a las salidas producidas por la ARNN del ejemplo.

Teorema 2. Toda máquina de estado finito tiene una ARNN con pesos enteros equivalente.

Demostración. Sea $\mathcal{M} = \langle E, S, Q, f, g, q_1 \rangle$ una máquina de estado finito con:

$E: \{e_1, e_2, \dots, e_p\}$ Alfabeto de entrada.

$S: \{s_1, s_2, \dots, s_v\}$ Alfabeto de salida.

$Q: \{q_1, q_2, \dots, q_w\}$ Conjunto de estados.

$f: Q \times E \rightarrow Q$ Función de cambio de estado.

$g: Q \times E \rightarrow S$ Función de salida.

$q_1 \in Q$ Estado inicial.

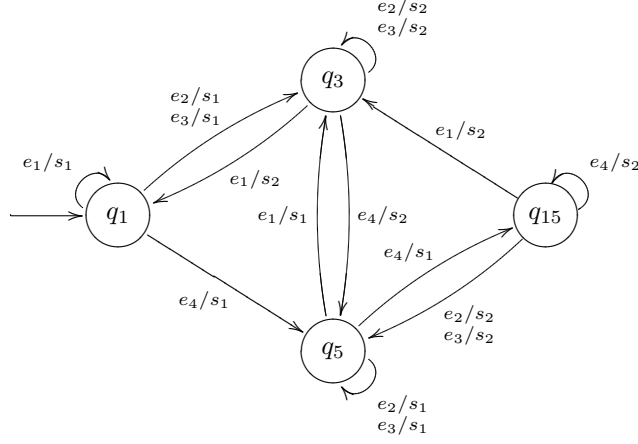


Figura 5: Diagrama de transición: conversión ARNN ejemplo 1.

La ARNN $\mathcal{N}$ con vector de activación $X(t)$, vector de entradas $U(t)$, matriz de pesos de conexiones A , matriz de pesos de entrada B , y vector de pesos constante C , equivalente a la máquina de estado finito $\mathcal{M}$ se define por (figura 6):

- Neuronas: por cada estado q_j se crean p neuronas, una neurona por cada símbolo de entrada, formandose una matriz de neuronas de dimensión $p \times w$, en cada instante de tiempo sólo una de estas neuronas tendrá valor de activación 1 (las demás tendrán valor de activación 0), el valor de activación 1 para la neurona x_{ij} es equivalente al evento de estar en el estado q_j con la entrada e_i en la máquina de estado finito $\mathcal{M}$. Además, por cada símbolo de salida s_i se debe crear una neurona x_k , de las cuales también sólo habrá una con valor de activación 1 en cada instante de tiempo (menos en $t = 1$ donde todas las salidas serán 0 debido a que $\mathcal{N}$ computará con retardo $r = 1$), si la neurona x_i tiene valor de activación 1 equivale a obtener la salida s_i .
- Entradas: por cada símbolo de entrada e_i habrá una entrada u_i , en cada instante de tiempo sólo una de estas entradas tendrá valor 1, de tal modo que la entrada $u_i = 1$ es equivalente a e_i . Además, se tendrá una entrada adicional u_{p+1} que servirá para activar la neurona x_{i1} que se corresponde con el evento inicial de la máquina $\mathcal{M}$ (estado inicial q_1 , símbolo de entrada e_i inicial), esta entrada tendrá valor 1 sólo en $t = 0$.
- Conexiones: habrá una conexión con peso $a_{kl,ij} = 1$ entre la neurona x_{ij} y la neurona x_{kl} si y sólo si $f(q_j, e_i) = q_l$ y habrá una conexión con peso $a_{k,ij} = 1$ entre la neurona x_{ij} y la neurona x_k si y sólo si $g(q_j, e_i) = s_k$.
- Pesos de entradas: las entradas u_i (menos la u_{p+1}) tendrán enlaces con peso $b_{ij,i} = 1$ hacia las neuronas x_{ij} . La neurona u_{p+1} tendrá enlaces con peso $b_{i1,p+1} = 1$ hacia las neuronas x_{i1} .
- Pesos constantes: las neuronas x_{ij} tendrán pesos constantes $c_{ij} = -1$ para que sólo se activen cuando reciban dos entradas (la correspondiente al cambio de estado y la correspondiente a la entrada). Las neuronas de salida x_k tendrán peso constante $c_k = 0$.

- Función de activación: función σ definida en la ecuación (2).

La ARNN $\mathcal{N}$ construida de ésta manera tiene el mismo comportamiento entrada-salida que la máquina de estado finito $\mathcal{M}$, pero $\mathcal{N}$ produce las salidas con un instante de tiempo de retardo que es lo que se demoran las entradas en afectar las neuronas de salida.

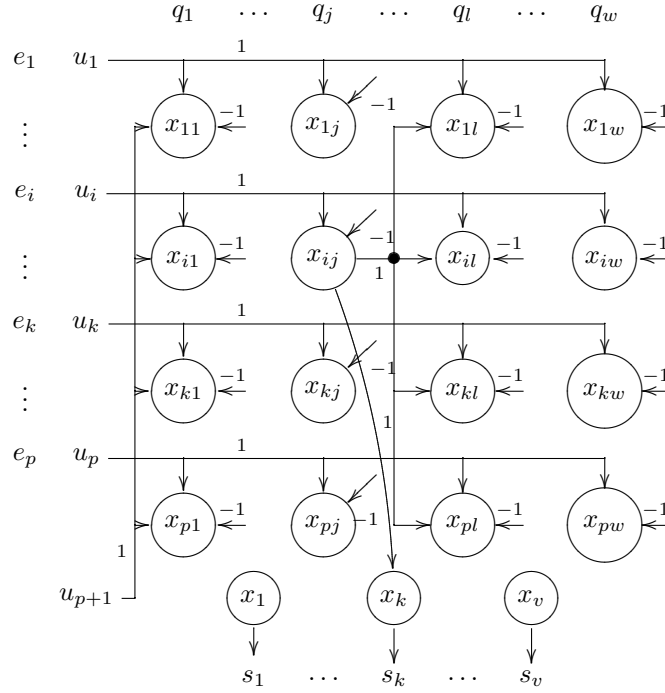


Figura 6: Conversión de máquina de estado finito a ARNN con pesos enteros. $f(q_j, e_i) = q_l$, $g(q_j, e_i) = s_k$.

□

Ejemplo 5. Para la máquina de estado finito definida en el ejemplo 2, teniendo en cuenta las siguientes enumeraciones de símbolos:

Estados: $q_1 = N$, $q_2 = A$.

Alfabeto de entrada: $e_1 = 00$, $e_2 = 01$, $e_3 = 10$, $e_4 = 11$.

Alfabeto de salida: $s_1 = 0$, $s_2 = 1$.

La ARNN equivalente está definida por (figura 7):

Neuronas: $X(t) = \{x_{11}, x_{12}, x_{21}, x_{22}, x_{31}, x_{32}, x_{41}, x_{42}, x_1, x_2\}$, donde x_1 y x_2 son neuronas de salida.

Matriz de conexiones entre las neuronas:

$$A = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

Entradas: $U(t) = \{u_1, u_2, u_3, u_4, u_5\}$.

Matriz de pesos de entradas:

$$B = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Vector de pesos constantes:

$$C = \begin{pmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ 0 \\ 0 \end{pmatrix}.$$

Para sumar los mismos valores del ejemplo 2 se tiene la siguiente computación:

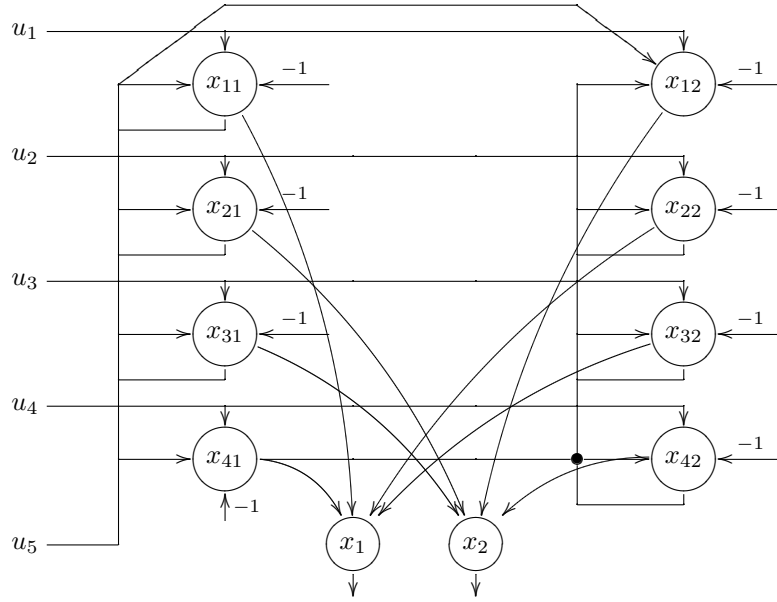


Figura 7: Diagrama de red: conversión máquina de estado finito del ejemplo 2, todas las conexiones tienen peso 1.

• Iteración 1:

$$X(1) = \sigma \left(\begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 1 \end{pmatrix} + \begin{pmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ 0 \\ 0 \end{pmatrix} \right) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

- Iteración 5:

$$X(5) = \sigma \left(\begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

Las salidas resaltadas con negrilla desde el tiempo $t = 2$ son 01, 01, 10, 01 que son equivalentes a los símbolos de salida 1, 1, 0, 1 obtenidos en el ejemplo 2.

5 Conclusiones

El resultado de restringir a los números enteros los valores que pueden tomar los pesos en una ARNN, con función de activación σ (ec. 2), es que las neuronas sólo pueden tomar un número finito de valores de activación (0 y 1). Esto hace que la ARNN sólo pueda estar en un número finito de ‘configuraciones’ debido a que el número de neuronas es finito, entendiéndose por configuración una combinación de los valores de activación de las neuronas, esta es la misma situación que se presenta en una máquina de estado finito y a esto se debe la equivalencia de los modelos.

El número de configuraciones que puede alcanzar una máquina también puede ser pensada como su capacidad de memoria, desde este punto de vista tanto las ARNNs con pesos enteros como las máquinas de estado finito tienen una capacidad finita de memoria.

La equivalencia computacional entre ARNNs con pesos enteros y máquinas de estado finito no sólo se debe al número finito de valores de activación de cada neurona, también se debe a que en la definición de ARNNs sólo permite un número finito de neuronas. Se puede pensar en definiciones de ARNNs donde el número de neuronas sea infinito y de este modo existe la posibilidad de obtener ARNNs con pesos enteros con un poder computacional mayor al de las máquinas de estado finito.

Agradecimientos

Este artículo fue financiado por la universidad EAFIT, bajo el proyecto de investigación número 817424.

Referencias

- [1] DECOROSO CRESPO. “Informática teórica. Primera parte”. Madrid: Departamento de Publicaciones de la Facultad de Informática, U. Politecnica de Madrid. (1983).

- [2] RAÚL GÓMEZ MARÍN Y ANDRÉS SICARD RAMÍREZ. “Informática teórica: Elementos propedeúticos”. Medellín: Fondo Editorial U. EAFIT (2001).
- [3] STEPHEN C. KLEENE. Representation of events in nerve nets and finite automata. En C. E. SHANNON Y J. MCCARTHY, editores, “Automata Studies”. Princenton University Press (1956).
- [4] W. S. MCCULLOCH Y W. PITTS. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics* **5**, 115–133 (1943).
- [5] MARVIN L. MINSKY. “Computation: finite and infinite machines”. Englewood Cliffs, N.J.: Prentice-Hall, Inc. (1967).
- [6] HAVA T. SIEGELMANN. “Neural networks and analog computation. Beyond the Turing limit”. Progress in Theoretical Computer Science. Boston: Birkhäuser (1999).